CFA INSTA - Architecture résiliente et exploitation des SI : M. Michaelis MAHOUTO

TP1 : AUDITER UN SERVICE ET TESTER SA RÉSISTANCE AUX PANNES

Étude de cas ShopAlto : diagnostic, maquette Web redondée et expériences de panne

Auteur	Landry TOUOTAP (travail individuel)
Dates des essais	12 octobre 2026 (essai 1) et 13 octobre 2026 (essais 2 à 4)
Poste de test	Windows, PowerShell, Docker Desktop 4.93.0 (moteur Docker 29.8.1, linux/amd64), Docker Compose v5.5.1
Images utilisées	nginx:stable-alpine (sha256:0985e772fb9f729e6fa0980da05fca5d9c468e870eed43071545afa9d2e27d94) ; haproxy:3.2-alpine, HAProxy 3.2.25 (sha256:5d97434a423c2533cfeb42874d45cf6d69a840b60334582dfbfd5008e94c80be)
Données	Chiffres de ShopAlto fictifs (sujet). Mesures de la maquette relevées par l'auteur, fichiers dans preuves/.

Synthèse. La proposition « deux serveurs pour ne plus avoir de panne » ne tient pas. Ce mois-ci, le portail a été indisponible 57 minutes pour un budget de 43,2 minutes (99,868 % au lieu de 99,9 %), et aucune des trois causes (maintenance de HV01, commutateur, base) n'aurait été évitée par un second serveur Web placé sur le même hôte. La maquette le confirme : deux serveurs Web derrière HAProxy tolèrent la perte d'*un* serveur avec une seule requête perdue, mais le service tombe entièrement si les deux serveurs ou le répartiteur s'arrêtent, et une sonde HTTP simple laisse passer une page erronée servie à un client sur deux. La priorité est de protéger les données (sauvegarde externalisée et testée), puis de séparer les domaines de panne (deuxième hôte, réseau doublé).

1. Analyse du dossier (Question 1 à Question 6)

Question 1 · Architecture actuelle et points uniques de défaillance

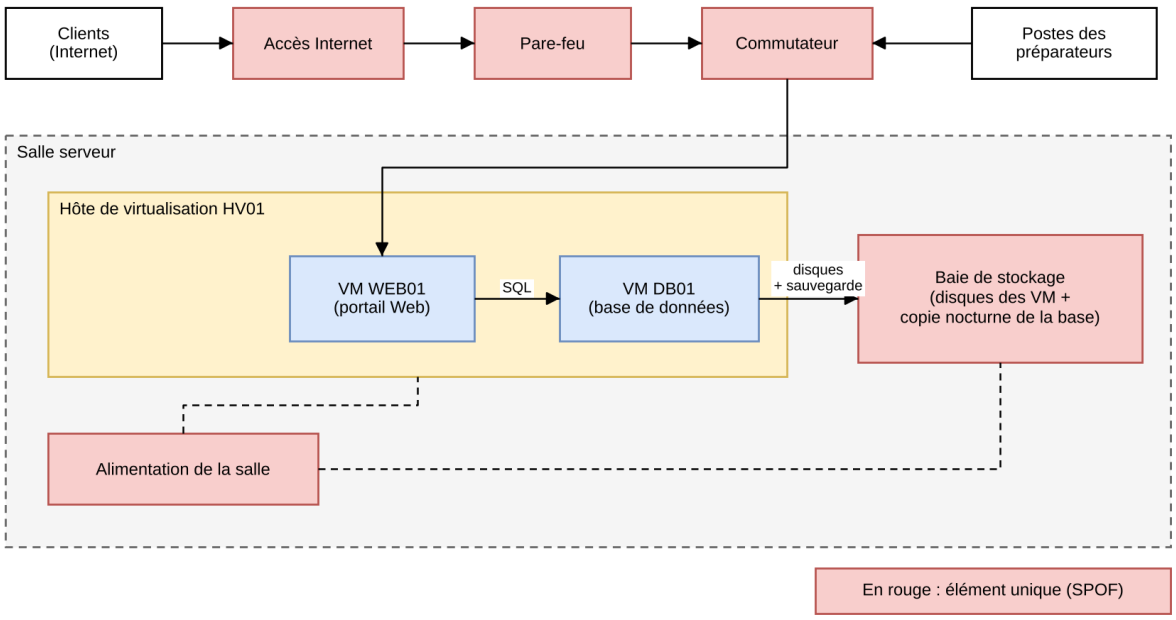


Figure 1 · Architecture actuelle de ShopAlto

SPOF ou dépendance commun et leur impact en cas de panne

- **Accès à internet unique** : Les clients ne joignent plus le portail ; les préparateurs continuent mais ne reçoivent plus de commandes.
- **Pare-feu unique** : Coupure totale du portail ; sa maintenance est elle aussi une interruption.
- **Commutateur unique** : Plus aucun flux entre réseau, HV01 et postes : cause de 27 min d'arrêt ce mois-ci.
- **Hôte HV01** (WEB01+DB01) : Une intervention ou une panne arrête les deux VM ensemble : 18 min d'arrêt pour maintenance.
- **Baie de stockage partagée :** Porte les disques des deux VM **et** la copie nocturne : sa perte arrête le service et détruit la sauvegarde.
- **Alimentation de salle unique :** Domaine de panne qui englobe réseau, hôte et baie : une coupure arrête toute la chaîne.
- **DB01 unique :** Sans base, aucune commande n'est possible même si le Web répond (12min d'arrêt ce mois-ci).
- **Sauvegarde jamais testée :** Risque de durabilité : rien ne prouve que la copie soit restaurable, ni dans quel délai

Question 2 · Criticité des fonctions

Rang	Fonction	Justification
1	Prise de commande	Bloque directement le chiffre d'affaires et l'organisation des 1 200 expéditions quotidiennes ; une commande perdue ne se rattrape pas toujours.
2	Consultation du catalogue	Réduit la navigation et prépare les ventes futures, mais aucune transaction n'est perdue sur le moment ; le client peut revenir.
3	Reporting du lendemain	Usage interne différé : quelques heures d'attente sont tolérables sans impact client.

La fonction qui peut tourner en mode dégradé : **la consultation du catalogue**. Si la base ou un service secondaire tombe, le catalogue reste consultable en lecture seule, depuis un cache ou une copie des pages produits, avec des prix et stocks « indicatifs ». Le client continue de naviguer ; seules les actions qui écrivent des données, comme passer commande, sont suspendues avec un message clair.

Question 3 · Disponibilité et budget d'arrêt

Grandeur	Formule	Résultat
Durée de la fenêtre	30 j × 24 h × 60 min	43 200 min
Budget d'arrêt	43 200 min × (1 − 0,999)	**43,2 min** (43 min 12 s)
Arrêt constaté	18 + 27 + 12 (périodes disjointes)	**57 min**
Disponibilité obtenue	(43 200 − 57) / 43 200	**99,868 %**
Écart au budget	57 − 43,2	**+13,8 min**, (soit 57 min ÷ 43,2 min = 1,319) soit **environ 132 %** du budget consommé

La cible de 99,9 % **n'est pas respectée**. À elle seule, la panne du commutateur consomme (27 min ÷ 43,2 min = 0,625) soit **62,5 %** du budget mensuel.

Question 4 · Valeur de commandes potentiellement affectée

Arrêt = 57 min = 0,95 h. Tentatives empêchées = 0,95 h × 40 tentatives/h = **38 tentatives**. Valeur potentiellement affectée = 38 × 80 € = **3 040 €**.

Ce montant mesure une **exposition**, pas une perte : une partie des clients a probablement réessayé plus tard ; toutes les tentatives n'auraient pas abouti (abandons, refus de paiement) ; le débit de 40 tentatives par heure est une hypothèse constante ; et aucune donnée sur les achats reportés n'est disponible. Les effets indirects (image, clients perdus durablement) ne sont pas chiffrés non plus.

Question 5 · « WEB02 sur HV01 = haute disponibilité complète » ?

Panne	Couverte ?	Explication
WEB01 (VM ou processus)	Oui, sous condition	Seulement avec un répartiteur et des contrôles de santé, qui deviennent un nouveau SPOF (démontré à l'essai 3).
HV01	Non	WEB01, WEB02 et DB01 partagent l'hôte : ils tombent ensemble.
Baie de stockage	Non	Les disques des trois VM et la sauvegarde sont sur la même baie.
Commutateur	Non	Il est en amont de tous les serveurs.

L'affirmation est fausse : seule la perte d'un processus Web est couverte. Aucun des trois incidents du mois n'aurait été évité, et DB01 reste unique.

Q06 · SLI de succès des commandes

- **Numérateur** : commandes confirmées au client et enregistrées en base, en moins de 3 s.
- **Dénominateur** : tentatives de commande valides reçues.
- **Fenêtre** : glissante de 30 jours.
- **Exclus** : erreurs imputables au client (panier invalide, formulaire incomplet), refus de paiement légitimes de la banque, trafic de test et robots.

La disponibilité temporelle (Question 3) compte des minutes quel que soit le trafic : une minute d'arrêt à 3 h du matin pèse autant qu'une minute en pointe. Le SLI par requêtes pondère par l'activité réelle et capte aussi les dégradations partielles (par exemple 5 % d'échecs pendant une heure), invisibles en minutes d'arrêt. La cible de 99,9 % est un **SLO** : un objectif interne, sans engagement contractuel ni pénalité. Ce n'est pas un SLA.

2. Maquette (Question 7 à Question 10)

Question 7 · Topologie annotée

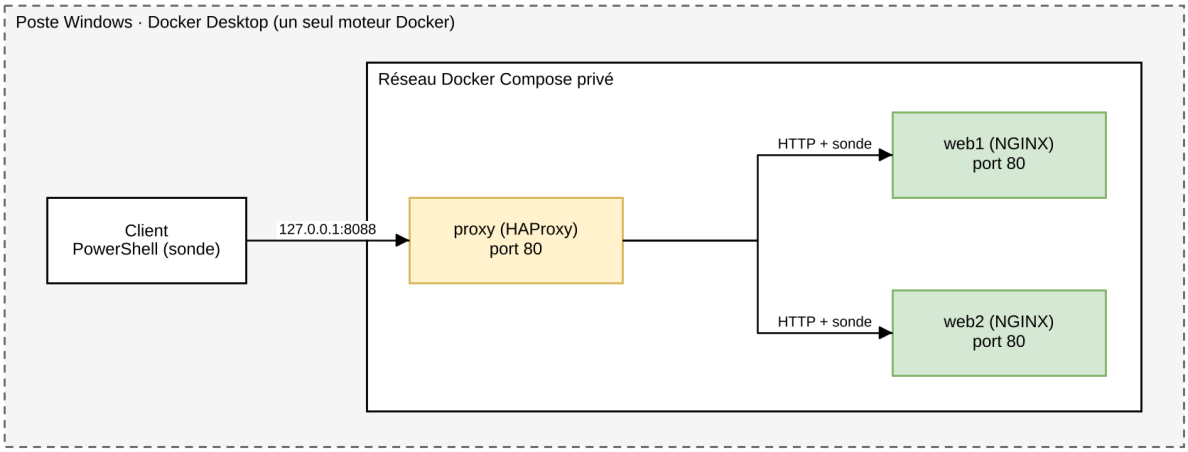


Figure 2 · Topologie de la maquette

Le client (la sonde PowerShell) n'atteint que HAProxy, via le seul port publié 127.0.0.1:8088, limité à la boucle locale. HAProxy joint web1 et web2 par leur nom DNS sur le réseau Compose privé, port 80 interne, et vérifie chaque seconde /health.html sur chacun. web1 et web2 n'ont pas besoin de port publié : seul le répartiteur doit être joignable, et publier les serveurs élargirait la surface d'exposition tout en permettant de contourner le répartiteur et ses contrôles. Tous les composants partagent le même poste, la même VM Docker et le même disque : c'est un seul domaine de panne.

Question 8 · Services démarrés et images utilisées

docker compose ps montre les trois services *Up* (capture preuves/q08_compose_ps.png), avec les références d'images données en première page (q08_images.png). Le montage :ro rend le contenu en lecture seule dans le conteneur : un processus compromis ou bogué ne peut pas le modifier, tandis qu'il reste modifiable depuis Windows (ce qui sert à l'essai 4).

Incident constaté au démarrage : docker compose up -d a affiché « Started » pour le proxy, qui s'est arrêté aussitôt : HAProxy refusait haproxy.cfg (« Missing LF on last line »). Après ajout du saut de ligne final, le proxy a démarré « Loading success ». Cela illustre la différence entre un conteneur *démarré* et un service *utilisable* ; de même, **depends_on** ne fixe qu'un ordre de démarrage.

Preuve : preuves/incident_haproxy_missing_LF.log.

Q09 · Répartition roundrobin

Les dix requêtes ont donné WEB1, WEB2, WEB1, WEB2…, soit cinq réponses par serveur en alternance stricte (q09_dix_requetes.png). roundrobin attribue chaque nouvelle requête au serveur sain suivant, à tour de rôle. Les deux serveurs traitent du trafic simultanément : la couche Web fonctionne en **actif-actif**.

Q10 · Paramètres des contrôles de santé

- **inter 1s** : un contrôle de /health.html par seconde et par serveur.
- **fall 2** : deux échecs consécutifs pour déclarer le serveur DOWN et cesser de lui envoyer du trafic.
- **rise 2** : deux succès consécutifs pour le déclarer UP à nouveau.

Pour un refus de connexion immédiat, la détection attendue est d'environ **1 à 2 s**, selon la phase des contrôles. Pour une panne lente (aucune réponse), chaque contrôle attend jusqu'à son délai, ici inter = 1 s faute de timeout check, avant d'être compté en échec : la détection est plus longue, et les requêtes clients envoyées entre-temps attendent jusqu'au timeout connect de 2 s. C'est ce qui a été observé : les arrêts de conteneurs ont produit des « Layer4 timeout » de 1 001 ms, et non des refus. Exiger plusieurs succès au retour (rise) évite de réintégrer un serveur qui redémarre ou qui oscille (« flapping »).

3. Expériences de panne (Question 11 à Question 16)

Méthode : sonde de 120 requêtes (une par seconde environ, délai maximal de 2 s) dans un terminal, injection des pannes dans un second terminal. Heures en heure de Paris ; les logs Docker, en UTC, ont été convertis (+2 h). Heures d'arrêt et de redémarrage relevées avec docker inspect (State.FinishedAt et StartedAt).

Q11 · Essai 1 : arrêt de web1 (12/10)

Heure	Source	Observation
16:57:52.49	sonde	Dernière réponse de WEB1 avant l'arrêt.
16:57:53.04	docker inspect	web1 arrêté.
16:57:54.62	sonde	Requête aiguillée vers web1 : ERREUR, délai expiré après 2 024 ms.
16:57:55.09	log HAProxy	*web1 is DOWN, reason: Layer4 timeout* ; « 1 active server left », « 1 sessions active, 0 requeued ».
16:57:57.66 → 16:58:31.82	sonde	Uniquement WEB2, aucun échec.
16:58:29.47	docker inspect	web1 redémarré.
16:58:30.84	log HAProxy	*web1 is UP, reason: Layer7 check passed, code 200* ; 2 serveurs actifs.
16:58:32.88	sonde	Première réponse WEB1 ; l'alternance reprend.

Avant l'arrêt, WEB1 et WEB2 alternent ; pendant, seul WEB2 répond ; après le redémarrage, web1 est réintégré automatiquement. **1 requête sur 120 a échoué.** Le log indique « 0 requeued » : la requête bloquée sur web1 n'a pas été rejouée vers web2, alors qu'un serveur sain était disponible. Preuves : essai1.csv, essai1_down_up.txt, essai1_heures_web1.txt, essai1_logs_proxy.txt et captures.

Question 12 · Délai de retrait et interruption côté client

- **Délai de retrait du backend** (arrêt → DOWN) : 16:57:53.04 → 16:57:55.09, soit **2,05 s**, cohérent avec deux contrôles en échec d'environ 1 s.
- **Délai de réintégration** (redémarrage → UP) : **1,37 s**, cohérent avec rise 2.
- **Interruption vue par le client** : 1 échantillon en échec ; 4,1 s entre deux réponses correctes (16:57:53.56 → 16:57:57.66) au lieu d'environ 1,1 s.

Les deux durées diffèrent parce que le client ne subit la panne que s'il est aiguillé vers web1 pendant la fenêtre de détection. Si aucune requête n'avait échoué, il faudrait écrire « aucun échec dans les échantillons » sans conclure à une interruption nulle : la sonde n'échantillonne qu'une requête par seconde et peut tomber, par le jeu du roundrobin, uniquement sur le serveur sain. Avec des centaines de clients, plusieurs requêtes auraient échoué.

Question 13 · Essai 2 : arrêt des deux serveurs Web (13/10)

Les deux serveurs sont arrêtés à 08:36:45.00. Pendant la panne, docker compose ps montre le proxy toujours *Up*, et curl.exe -i http://127.0.0.1:8088 renvoie HTTP/1.1 503 Service Unavailable avec le message « No server is available to handle this request ». Les logs contiennent [ALERT] backend 'serveurs_web' has no server available!.

Phase	Heure	Observation
Avant détection	08:36:45.27 et 08:36:48.29	2 requêtes en échec après 2 s d'attente : envoyées vers des serveurs déjà arrêtés.
Détection	08:36:47.05 / 08:36:48.47	web2 puis web1 déclarés DOWN (2,05 s et 3,46 s après l'arrêt).
Panne franche	08:36:51 → 08:37:50.46	59 réponses 503 immédiates (3 à 14 ms).
Retour	08:37:48.89 / 08:37:49.62	web2 puis web1 redémarrés ; web1 UP à 08:37:50.70, web2 UP à 08:37:52.26.
Reprise client	08:37:51.48	Première réponse correcte.

61 échecs sur 120, environ **66 s** sans service. Un processus disponible (HAProxy démarré et répondant) n'est pas un portail disponible : une supervision qui vérifierait seulement que le proxy tourne aurait tout affiché au vert. Deux serveurs Web qui tombent ensemble, comme deux VM sur HV01, ne protègent de rien.

Question 14 · Essai 3 : arrêt du répartiteur (13/10)

Le proxy est arrêté à 08:56:29.32. web1 et web2 restent *Up* (capture essai3_compose_ps.png), mais curl.exe échoue : « Failed to connect to 127.0.0.1:8088 after 2043 ms ». Contrairement à l'essai 2, personne ne répond, même pour refuser. Le client ne joint plus les serveurs parce qu'ils n'ont aucun port publié : le répartiteur est le seul chemin. La sonde relève 12 échecs, de 08:56:29.02 (connexion coupée pendant l'arrêt) à 08:57:00.27, puis une réponse correcte à 08:57:02.30, soit environ **33 s** sans service.

Fragilité démontrée : le répartiteur est un SPOF ; doubler les serveurs sans doubler ce qui est devant eux n'apporte pas la haute disponibilité. **Principe de correction :** deux répartiteurs sur deux hôtes distincts partageant une IP virtuelle (VRRP, par exemple avec keepalived). Si le répartiteur actif tombe, le second reprend l'adresse et le client continue d'utiliser la même entrée.

Question 15 · Essai 4 : HTTP correct, contenu incorrect (13/10)

De 09:02:10 à 09:02:44, web1/index.html est remplacé par « MAINTENANCE » ; health.html reste intact. Résultat : **0 échec HTTP** et **15 réponses 200 au contenu faux** (HttpOK = True, MetierOK = False), une requête sur deux, de 09:02:14.93 à 09:02:44.82. **HAProxy ne retire pas web1** : aucun message DOWN dans les logs. Il ne contrôle que /health.html, qui répondait toujours 200 ; le chemin

servi aux clients (/) n'est jamais vérifié. Le contrôle prouve que le serveur est vivant, pas qu'il rend le bon service.

Sonde plus représentative : au minimum, contrôler la page réellement servie et son contenu (GET / avec un texte attendu, par exemple http-check expect string SHOPALTO). Pour un portail de commandes : un point de santé applicatif qui vérifie la lecture du catalogue et l'écriture en base, sans retirer tous les serveurs Web à la moindre lenteur de la base, complété par une sonde synthétique externe qui passe régulièrement une commande de test de bout en bout et alimente le SLI de Q06.

Q16 · Tableau des expériences

N°	Action et heure	Symptôme	Résultat attendu	Résultat observé	Échecs HTTP	Échecs contenu	Reprise	Preuves
1	stop web1, 12/10 16:57:53	web1 injoignable	Retrait en ≈ 2 s, service assuré par web2, quelques échecs possibles	DOWN à +2,05 s ; seul WEB2 répond ; UP 1,37 s après redémarrage	1 / 120	0	16:58:32.88	essai1.csv, logs, heures
2	stop web1 web2, 13/10 08:36:45	plus aucun backend	503 renvoyé par HAProxy, proxy toujours démarré	2 délais expirés puis 59 × 503 ; proxy Up ; ≈ 66 s sans service	61 / 120	0	08:37:51.48	essai2.csv, logs, heures, curl
3	stop proxy, 13/10 08:56:29	point d'entrée absent	Aucune connexion possible, serveurs Web intacts	web1/web2 Up ; connexion impossible ; ≈ 33 s sans service	12 / 120	0	08:57:02.30	essai3.csv, logs, heures, curl
4	index.html de web1 = MAINTENANCE, 13/10 09:02:10 → 09:02:44	page erronée sur web1	Réponses 200 au contenu faux, web1 non retiré	1 réponse sur 2 fausse ; aucun DOWN	0 / 120	15	Dès la restauration	essai4.csv, captures

Écarts et limites de mesure :

- La sonde vise 127.0.0.1 au lieu de localhost : avec localhost, Windows tentait d'abord l'adresse IPv6 ::1, non publiée, et la première requête d'un test sans panne échouait. Ce délai côté client aurait faussé les mesures.
- L'essai 2 a été réalisé deux fois : la première fois, les serveurs ont été redémarrés après la fin de la sonde (arrêt visible dans les logs à 08:34:22). Seul le second essai est retenu.
- Le nombre d'échecs ne mesure pas la durée : à l'essai 3, chaque échec prenait 2 s plus 1 s de pause, soit une mesure toutes les 3 s environ. Les durées sont calculées à partir des heures.

- ❖ Horloges Windows et Docker comparées : écart inférieur à 1 s. Les délais sont donc précis à environ 1 s près ; cela explique une réponse WEB2 apparemment antérieure de 0,8 s à son retour UP (essai 2).
- ❖ Essai 4 : heures de début et de fin approximatives, les commandes Get-Date et Set-Content ayant été tapées séparément.
- ❖ TP réalisé seul (autorisé) : les rôles d'opérateur (injection des pannes) et d'observateur (sonde) ont été tenus par la même personne, dans deux fenêtres PowerShell ; l'échange de rôles prévu après l'essai 2 était donc sans objet.
- ❖ Tout tourne sur un seul poste : la maquette ne teste ni l'hôte, ni le stockage, ni le réseau, ni la base. MetierOK vérifie un texte de page, pas une commande réelle.

4. Proposition (Question 17 à Question 20)

Question 17 · Architecture cible à deux hôtes physiques

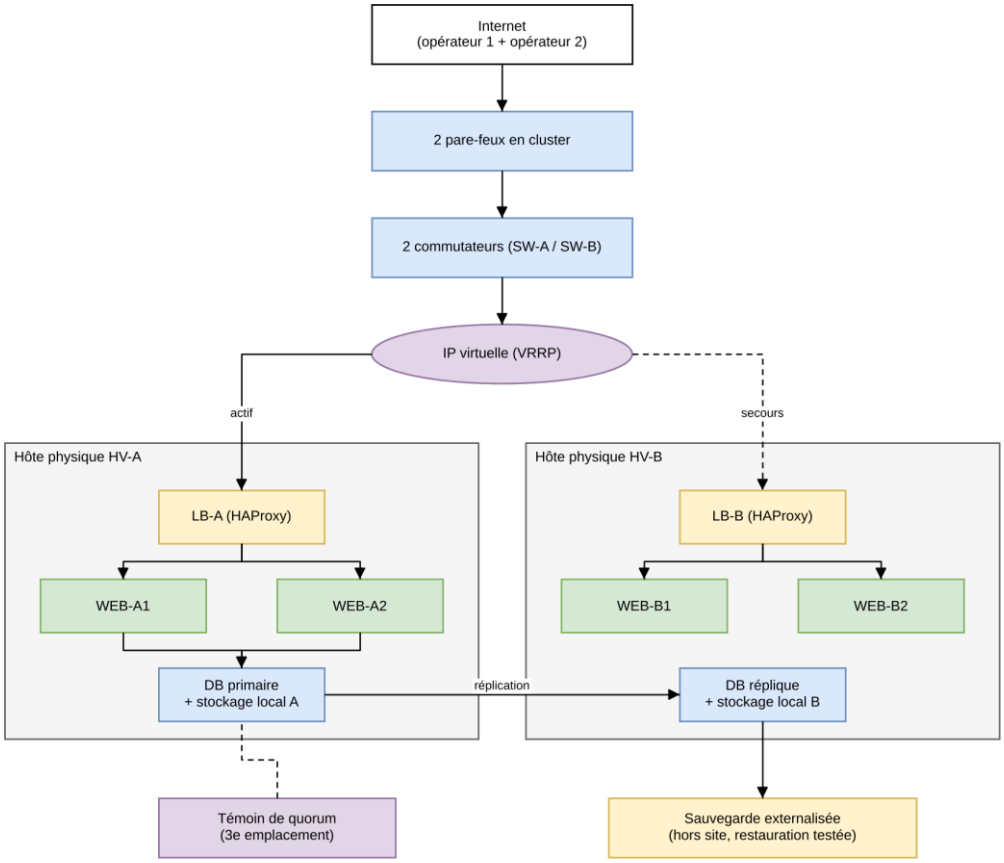


Figure 3 · Architecture cible à deux hôtes physiques

- ❖ **Domaines de panne :** hôte HV-A et hôte HV-B, chacun avec double alimentation sur deux PDU distinctes et son propre stockage. On supprime la baie commune.
- ❖ **Entrée :** deux pare-feux en cluster, deux commutateurs avec liens agrégés, deux répartiteurs HAProxy partageant une IP virtuelle (VRRP/keepalived).
- ❖ **Web :** deux serveurs Web par hôte, pour que chaque hôte supporte seul la pointe de 120 req/s (2 × 80 = 160 req/s), avec des contrôles de santé portant sur le contenu.
- ❖ **Base :** primaire sur HV-A, réplique sur HV-B, témoin de quorum sur un 3e emplacement ; promotion contrôlée avec fencing pour éviter deux écrivains.

- ❖ **Données et supervision :** sauvegarde externalisée hors baie et hors site, copie immuable, restauration testée et chronométrée ; sonde métier de bout en bout.

Risques encore non couverts : sinistre de la salle unique (incendie, coupure électrique générale) ; accès Internet unique si un second opérateur n'est pas souscrit ; corruption logique ou suppression accidentelle répliquée instantanément vers la réplique ; rançongiciel atteignant les deux hôtes ; erreur de configuration déployée simultanément sur les deux nœuds.

Question 18 · Dimensionnement

Situation	Calcul	Capacité	Pointe de 120 req/s
2 serveurs, normal	2 × 80	160 req/s	Couverte (75 % de charge)
2 serveurs, un perdu	1 × 80	80 req/s	**Non couverte** : déficit de 40 req/s
3 serveurs, un perdu	2 × 80	160 req/s	Couverte (75 % de charge)

La redondance à deux serveurs tolère la panne mais pas la charge de pointe. Il faut $(N - 1) \times 80 \geq 120$, donc $N - 1 \geq 1{,}5$, soit **N = 3 serveurs** (N+1) : 40 req/s par serveur en pointe, 60 req/s après une perte. Si l'on raisonne par hôte comme en Q17, il faut 120 req/s sur chaque hôte, donc deux serveurs Web par hôte (quatre au total).

Question 19 · Trois améliorations priorisées

N°	Amélioration	Risque traité	Contrainte	Responsable	Test d'acceptation
1	Sauvegarde externalisée (hors baie, copie immuable) et test de restauration	Perte définitive des commandes, irréversible contrairement à un arrêt	Stockage externe, fenêtre de test, chiffrement	Administrateur système ; DSI pour le budget	Restauration complète sur un environnement isolé, durée et point de reprise mesurés et conformes
2	Second hôte physique, réplication de la base, deux répartiteurs avec IP virtuelle	Arrêt de HV01, de la baie ou de DB01 (30 min ce mois-ci) ; SPOF du répartiteur	Coût, licences, procédure de bascule de la base	DSI (financement), équipe infrastructure	Arrêt d'un hôte en heures creuses : SLI de commande ≥ 99,9 % sur la période, aucune commande perdue
3	Réseau doublé (2 commutateurs, pare-feux en cluster) et sonde métier	Panne du commutateur (27 min) et pannes silencieuses comme l'essai 4	Recâblage, LACP et VRRP, fenêtre de maintenance	Équipe réseau ; exploitation pour la sonde	Débrancher un commutateur sans échec de la sonde métier ; page erronée détectée et retirée en moins de 5 s

Justification de l'ordre : une panne se rattrape, une perte de données non ; la sauvegarde est aussi l'action la moins chère. Le second hôte vient ensuite car il traite trois des SPOF majeurs. Le réseau, bien que cause de l'incident le plus long, est placé en troisième car sa correction est moins coûteuse et peut être menée en parallèle.

Q20 · Avis à la DSI

La maquette démontre que deux serveurs Web derrière un répartiteur absorbent la perte d'un serveur : retrait en 2 s, une requête perdue sur 120. Elle démontre aussi leurs limites : perte des deux serveurs = 61 échecs sur 120 avec un proxy pourtant démarré ; perte du répartiteur = 33 s d'arrêt avec deux serveurs sains ; page erronée servie à un client sur deux sans aucune alerte. Elle ne démontre rien sur l'hôte, le stockage, le réseau, la base ni le paiement, puisque tout tourne sur un seul poste. Deux serveurs Web ne protègent pas les données : ils ne stockent aucune commande. La base reste unique et sa seule copie est sur la même baie, sans test de restauration ; une redondance de calcul n'est pas une durabilité. Prochaine action à financer : une sauvegarde externalisée avec un test de restauration chronométré, puis un second hôte physique.

Annexe : Contenu de l'archive et index des preuves

Élément	Contenu
TP1_rapport	Le présent rapport.
compose.yaml, haproxy.cfg, sonde.ps1	Configuration et sonde utilisées.
web1/, web2/	index.html et health.html de chaque serveur.
schemas/	Les trois schémas (Q01, Q07, Q17), en image et en fichier diagrams.net modifiable.
preuves/essai1.csv … essai4.csv	Mesures brutes des quatre essais (120 requêtes chacun).
preuves/*_logs_proxy.txt, essai1_down_up.txt	Extraits de logs HAProxy horodatés (UTC).
preuves/essai1_heures_web1.txt, essai2_heures.txt, essai3_heures.txt	Heures d'arrêt et de redémarrage relevées par docker inspect.
preuves/incident_haproxy_missing_LF.log	Incident de démarrage du proxy (Q08).
preuves/*.png	Captures : compose ps, images, dix requêtes, chronologies, curl 503, bilans.